COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	7/16/2026	**DATE SUBMITTED:**	7/17/2026
NAME:	Vince Joseph Garcia	**INSTRUCTOR:**	Engr. Jlmlord Queajdo

ACTIVITY NO. _2.1__ [Arrays_Pointers_and Dynamic Memory Allocation]

1. PROCEDURE OUTPUT

HEADER

```
#include <iostream>
#include <string>

class Student{
    private:
        std::string studentName;
        int studentAge;

    public:
        //constructor
        Student(std::string newName ="John Doe", int newAge=18){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };

        //destructor
        ~Student(){
            std::cout << "Destructor Called." << std::endl;
        };

        //copy constructor
        Student(const Student &copySTudent){
            std::cout << "Copy Constructor Called." << std::endl;
            studentName = copySTudent.studentName;
            studentAge = copySTudent.studentAge;
        };
            //Display Attributes
            void printDetails(){
                std::cout<<this->studentName<<""<<this->studentAge<<std::endl;
            }
};
```

1ST DRIVER PROGRAM

```
34  int main(){
35      Student student1("Garcia", 19);
36      Student student2(student1);
37      Student student3;
38      student3 = student2;
39
40      return 0;
41  }
```

OUTPUT

```
Constructor Called.
Copy Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

OBSERVATION

This program shows how C++ creates, copies, and destroys objects in memory. first, **student1** is created with custom info, **student2** copies it (which triggers the copy constructor), and **student3** is made using default values. The tricky part is **student3** = **student2**,is just a basic assignment, not a new object being built, so it silently copies the data without printing anything. Finally, when the program ends, the stack cleans itself up in reverse order **(last in, first out)**, which is why you see the three destructors fire from newest (**student3**) to oldest (**student1**).

2nd DRIVER PROGRAM

```
int main(){
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

OUTPUT

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

OBSERVATION

This code shows how C++ handles arrays of objects. By creating the studentList array with a size of 5, the program automatically fires the default constructor 5 times to build each empty student in the array using those default fallback values **("John Doe", 18)**. Note that the name and age arrays are completely separate, so they don't actually interact with our students here. When **main()** ends, the entire student array goes out of scope, which triggers 5 destructor calls in reverse order to clean all those objects out of memory.

3rd DRIVER PROGRAM

```
int main(){
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    for(int i = 0; i < j; i++){//loop a
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;
    }

    for(int i = 0; i < j ; i++){//loop b
        studentList[i].printDetails();
    }

    return 0;
}
```

LOOP A

```
for(int i = 0; i < j; i++){//loop a
    Student *ptr = new Student(namesList[i], ageList[i]);
    studentList[i] = *ptr;
}
```

OBSERVATION

This loop shows how we can dynamically create objects on the heap and assign their values to our stack-allocated array. For each iteration in **Loop A**, a new Student is built using the new keyword, which triggers the parameterized constructor with names and ages from our lists. We then dereference the pointer ptr and copy its values directly into the array. The tricky part here is that we never call delete **ptr** inside the loop. This creates a memory leak because those heap-allocated student objects are never destroyed in memory. That is why we see 10 "Constructor Called" prints (5 from the initial empty array initialization plus 5 from the loop) but only 5 "Destructor Called" prints when **main()** ends and cleans up the stack array.

LOOP B

```
for(int i = 0; i < j ; i++){//loop b
    studentList[i].printDetails();
}
```

OBSERVATION

This loop shows how we easily access and display the stored data from our array of objects. In **Loop B**, we iterate through studentList and call the **printDetails()** member function for each index. Since the data from our dynamically created heap objects was successfully copied over to our array back in **Loop A**, this loop cleanly prints each student's customized name and age (such as "Carly15") directly to the console. Because this loop only reads and outputs the existing values without creating, copying, or destroying any objects, it executes without triggering any constructors or destructors

OUTPUT

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Carly15
Freddy16
Sam18
Zack19
Cody16
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers

Jenna's Grocery List
Apple PHP 10 x7
Banana PHP 10 x8
Broccoli PHP 60 x12
Lettuce PHP 50 x10

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.
Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.
Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.
Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.
Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

FULL CODE:

```cpp
#include <iostream>
#include <string>

class Fruit {
private:
    std::string name;
    double price;
    int quantity;

public:
    Fruit(std::string n = "", double p = 0, int q = 0) : name(std::move(n)), price(p), quantity(q) { std::cout
        << "Fruit Constructor Called." << std::endl; }

    ~Fruit() { std::cout << "Fruit Destructor Called." << std::endl; }

    Fruit(const Fruit &c) : name(c.name), price(c.price), quantity(c.quantity)
        {std::cout << "Fruit Copy Constructor Called." << std::endl; }

    Fruit& operator=(const Fruit &c) { std::cout << "Fruit Copy Assignment Called." << std::endl; if (this != &c)
        { name = c.name; price = c.price; quantity = c.quantity; } return *this; }

    double calculateSum() { return this->price * this->quantity; }

    void printDetails() { std::cout << "Fruit: " << this->name << " PHP "
        << this->price << " x" << this->quantity << std::endl; }
};

```

```cpp
class Vegetable {
private:
    std::string name;
    double price;
    int quantity;

public:
    Vegetable(std::string n = "", double p = 0, int q = 0) : name(std::move(n)), price(p), quantity(q)
        { std::cout << "Vegetable Constructor Called." << std::endl; }

    ~Vegetable() { std::cout << "Vegetable Destructor Called." << std::endl; }

    Vegetable(const Vegetable &c) : name(c.name), price(c.price), quantity(c.quantity)
        { std::cout << "Vegetable Copy Constructor Called." << std::endl; }

    Vegetable& operator=(const Vegetable &c) { std::cout << "Vegetable Copy Assignment Called." << std::endl; if (this != &c)
        { name = c.name; price = c.price; quantity = c.quantity; } return *this; }

    double calculateSum() { return this->price * this->quantity; }

    void printDetails() { std::cout << "Vegetable: " << this->name << " PHP " << this->price << " x"
        << this->quantity << std::endl; }
};
```

```cpp

//total sum
double TotalSum(Fruit* f[], int fSize, Vegetable* v[], int vSize) {
    double total = 0;
    for (int i = 0; i < fSize; i++) if (f[i]) total += f[i]->calculateSum();
    for (int i = 0; i < vSize; i++) if (v[i]) total += v[i]->calculateSum();
    return total;
}
//main driver
int main() {

    Fruit* fruitList[2] = { new Fruit("Apple", 10, 7), new Fruit("Banana", 10, 8) };
    Vegetable* vegList[2] = { new Vegetable("Broccoli", 60, 12), new Vegetable("Lettuce", 50, 10) };

    std::cout << "\n--- Jenna's Grocery List ---" << std::endl;
    for (int i = 0; i < 2; i++) if (fruitList[i]) fruitList[i]->printDetails();
    for (int i = 0; i < 2; i++) if (vegList[i]) vegList[i]->printDetails();

    //total Sum
    std::cout << "\nTotal Sum: PHP " << TotalSum(fruitList, 2, vegList, 2) << std::endl;

    std::cout << "\n--- Deleting Lettuce ---" << std::endl;
    delete vegList[1];
    vegList[1] = nullptr;

    std::cout << "Updated Total Sum: PHP " << TotalSum(fruitList, 2, vegList, 2) << std::endl << std::endl;

```

```cpp
    // Memory Cleanup
    for (int i = 0; i < 2; i++) delete fruitList[i];
    for (int i = 0; i < 2; i++) delete vegList[i];

    return 0;
}
```

PROBLEM 1:

```cpp
class Fruit {
private:
    std::string name;
    double price;
    int quantity;

public:
    Fruit(std::str (const char [26])"Fruit Constructor Called." d::move(n)), price(p), quantity(q) { std::cout
        << "Fruit Constructor Called." << std::endl; }

    ~Fruit() { std::cout << "Fruit Destructor Called." << std::endl; }

    Fruit(const Fruit &c) : name(c.name), price(c.price), quantity(c.quantity)
        {std::cout << "Fruit Copy Constructor Called." << std::endl; }

    Fruit& operator=(const Fruit &c) { std::cout << "Fruit Copy Assignment Called." << std::endl; if (this != &c)
        { name = c.name; price = c.price; quantity = c.quantity; } return *this; }

    double calculateSum() { return this->price * this->quantity; }

    void printDetails() { std::cout << "Fruit: " << this->name << " PHP "
        << this->price << " x" << this->quantity << std::endl; }
};
```

```cpp
class Vegetable {
private:
    std::string name;
    double price;
    int quantity;

public:
    Vegetable(std::string n = "", double p = 0, int q = 0) : name(std::move(n)), price(p), quantity(q)
        { std::cout << "Vegetable Constructor Called." << std::endl; }

    ~Vegetable() { std::cout << "Vegetable Destructor Called." << std::endl; }

    Vegetable(const Vegetable &c) : name(c.name), pr .quantity)
        { std::cout << "Vegetable Copy Constructor C
    std::ostream std::cout
    Linked to standard output

    Vegetable& operator=(const Vegetable &c) { std::cout << "Vegetable Copy Assignment Called." << std::endl; if (this != &c)
        { name = c.name; price = c.price; quantity = c.quantity; } return *this; }

    double calculateSum() { return this->price * this->quantity; }

    void printDetails() { std::cout << "Vegetable: " << this->name << " PHP " << this->price << " x"
        << this->quantity << std::endl; }
};
```

OBSERVATION:

This section defines the class structures required to manage the inventory. By implementing the constructor, destructor, copy constructor, and copy assignment operator, we ensure the program handles object lifecycle events predictably. Defining calculateSum and printDetails within each class keeps the logic encapsulated, allowing us to easily process individual item data such as **"Apple PHP 10 x7"**

PROBLEM 2:

```
Fruit* fruitList[2] = { new Fruit("Apple", 10, 7), new Fruit("Banana", 10, 8) };
Vegetable* vegList[2] = { new Vegetable("Broccoli", 60, 12), new Vegetable("Lettuce", 50, 10) };

std::cout << "\n--- Jenna's Grocery List ---" << std::endl;
for (int i = 0; i < 2; i++) if (fruitList[i]) fruitList[i]->printDetails();
for (int i = 0; i < 2; i++) if (vegList[i]) vegList[i]->printDetails();
```

OBSERVATIONS:

This demonstrates how to store multiple objects in an array structure. By using pointers to create the fruitList and **vegList**, we manage each grocery item as a distinct instance. When these are initialized, the constructor for each fruit and vegetable is called, successfully placing the specific items (like "Broccoli" and "Lettuce") into their respective arrays, allowing for a clean display of all items when we iterate through them

PROBLEM 3:

```
//total sum
double TotalSum(Fruit* f[], int fSize, Vegetable* v[], int vSize) {
    double total = 0;
    for (int i = 0; i < fSize; i++) if (f[i]) total += f[i]->calculateSum();
    for (int i = 0; i < vSize; i++) if (v[i]) total += v[i]->calculateSum();
    return total;
}
```

```
//total Sum
std::cout << "\nTotal Sum: PHP " << TotalSum(fruitList, 2, vegList, 2) << std::endl;

std::cout << "\n--- Deleting Lettuce ---" << std::endl;
delete vegList[1];
vegList[1] = nullptr;

std::cout << "Updated Total Sum: PHP " << TotalSum(fruitList, 2, vegList, 2) << std::endl << std::endl;
```

OBSERVATION:

This function provides a centralized way to aggregate data across different object types. By iterating through both the **fruitList** and **vegList** and calling **calculateSum**() for each valid item, we can derive the grand total cost. This modular approach proves useful because it separates the mathematical calculation from the storage logic, ensuring the total sum is accurate regardless of the quantity of items added.

PROBLEM 4:

```
std::cout << "\n--- Deleting Lettuce ---" << std::endl;
delete vegList[1];
vegList[1] = nullptr;
```

OBSERVATION:

This demonstrates the manual management of dynamic memory. By calling delete on the specific index where "Lettuce" was stored, we immediately trigger that object's destructor to clean up its resources. Setting the pointer to nullptr afterward is a crucial step to prevent dangling pointers, ensuring that when the TotalSum is recalculated, the program correctly identifies that the item has been removed from the list.

CODE OUTPUT:

```
Fruit Constructor Called.
Fruit Constructor Called.
Vegetable Constructor Called.
Vegetable Constructor Called.

--- Jenna's Grocery List ---
Fruit: Apple PHP 10 x7
Fruit: Banana PHP 10 x8
Vegetable: Broccoli PHP 60 x12
Vegetable: Lettuce PHP 50 x10

Total Sum: PHP 1370

--- Deleting Lettuce ---
Vegetable Destructor Called.
Updated Total Sum: PHP 870

Fruit Destructor Called.
Fruit Destructor Called.
Vegetable Destructor Called.
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **QUILLBOT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, paraphrase, and spell check**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Quilbot was used to help me convey or fix how i worded my observations of my work**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or

completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.